

DEDAN KIMATHI UNIVERSITY OF TECHNOLOGY
CENTRE FOR DATA SCIENCE AND ARTIFICIAL INTELLIGENCE (DSAIL)

JasinaHub

Architecture and Developer Guide for Scalable Speech and Text Data Collection in Low-Resource African Languages

Braxton Mandara¹ Mary Kariuki² Ciira wa Maina³

^{1,2,3} Dedan Kimathi University of Technology,
Centre for Data Science and Artificial Intelligence (DSAIL),
P.O. Box 657 – 10100, Nyeri, Kenya

¹braxton.mandara@dkut.ac.ke

²mary.kariuki@dkut.ac.ke

³ciira.maina@dkut.ac.ke

ABSTRACT

Curated speech data and accessible collection tools remain scarce for many African languages, constraining automatic speech recognition (ASR) development. JasinaHub is a mobile-responsive web platform that integrates spontaneous and read-speech collection, transcription, quality review, and governed export. Role-based access control, atomic transcription reservation, interrupted-upload recovery, and pseudonymised export support mobile participation, intermittent connectivity, and concurrent transcription.

An operational pilot involved 25 consenting adults representing four regional Kikuyu dialects and 369 health-related prompts. Participants submitted 2,275 recordings totalling 16 h 18 min. Administrative review retained 1,789 recordings totalling 13 h 16 min; all retained recordings were transcribed, yielding 1,789 completed speech–transcription pairs. The retained collection represents 78.6% of submitted recordings and 81.4% of recorded duration.

The pilot demonstrated the collection-to-export workflow under practical deployment conditions, but it was not a formal usability study or an ASR evaluation. Its small sample, single language, and health-domain focus limit generalisation. The corpus is being used to develop *Jasina*, a separate Kikuyu ASR model. JasinaHub is released under the MIT licence and can be configured for additional languages and domains; model development remains outside the platform boundary.

Keywords: Speech corpus, Low-resource languages, Automatic Speech Recognition, Progressive Web Application, PostgreSQL Row-Level Security.

List of Figures

1	Logical interaction between the JasinaHub client, managed backend services, persistent storage, and exported dataset.	4
2	End-to-end recording, moderation, transcription, validation, and export workflow.	5
3	Use case diagram showing the three actors (Volunteer, Transcriber, Administrator) and the fourteen primary use cases of the JasinaHub system.	7
4	Conceptual entity relationships for the core recording and transcription workflow; release migrations remain authoritative.	9
5	Atomic transcription claim and submission workflow.	10
6	Recording, local recovery, object upload, and database-persistence sequence.	11
7	Mobile recording interface showing the active prompt, timer, progress, and skip control.	11
8	Administrator overview of collection statistics and review queues; screenshot values are not authoritative pilot totals.	15
9	Transcription workspace with guidance, prompt, audio controls, and verbatim text entry.	16
10	Administrator configuration of feedback questions and response types.	17
11	Administrator test runner showing point-in-time suite results.	19

List of Tables

1	Document and code metadata.	1
2	Comparison with related speech-collection platforms and workflows. A dash denotes a capability not reported in the cited publication, not evidence that it is absent from later software.	3
3	Primary workflow contracts.	4
4	Minimum authorization matrix.	6
5	JasinaHub public-schema overview.	8
6	Core release invariants and enforcement requirements.	8
7	Transcription lease lifecycle.	10
8	Recording and recovery failure handling.	12
9	Documented RPC behaviour and required failure handling; exact signatures require release verification.	12
10	Recording rejection criteria.	14
11	JasinaHub pilot deployment and quality-review outcomes.	17
12	Required automated test layers.	18

Listings

1	Reference clean-checkout installation workflow.	2
2	Reference local backend workflow.	2
3	Reference frontend verification workflow.	2

Contents

1 Document Orientation and Control	1
2 Introduction	1
3 System Scope, Assumptions, and Objectives	1
4 Developer Quick Start and Reproducibility	2
5 Related Work and Literature Review	3
6 System Architecture and Workflows	3
7 System Roles and Authorisation Model	6
7.1 Volunteer (Recording Contributor)	6
7.2 Transcriber	6
7.3 Administrator	6
7.4 Authorization Matrix	6
8 Development and Contribution Approach	7
9 Database Design and Domain Invariants	8
10 Concurrency and Consistency Management	9
11 Voice Recording and Recovery	10
11.1 Audio Contract	11
11.2 Interrupted-Upload Recovery	12
11.3 Worked Lifecycle	12
12 Backend Interface Reference	12
12.1 Core RPC Contracts	12
12.2 Edge Function Contracts	13
13 Storage and Data Lifecycle	13
14 Security, Privacy, and Consent	13
14.1 Privacy and Consent	14
15 Quality Assurance and Dataset Export	14
15.1 Recording Review	14
15.2 Transcription and Validation	14
15.3 Dataset Export	15
15.4 In-Product Feedback	15
16 Pilot Deployment and Dataset Statistics	16
17 Impact and Reuse Potential	17
17.1 Limitations	18
18 Observability, Testing, and Release Evidence	18
18.1 Operational Visibility	18
18.2 Test Strategy	18
18.3 Interface and Release Evidence	19
19 Future Extensions	20
20 Conclusion	20
21 Project Attribution and Declarations	20

22 Acknowledgments	20
A Architecture Decision Summary	21
B Glossary	21

1. Document Orientation and Control

This handbook describes the architecture, development workflow, security baseline, quality controls, and pilot evidence for JasinaHub release v1.0.

Table 1: Document and code metadata.

Item	Value
Document version and date	1.3; 25 September 2026
Software release	v1.0
Maintainers	Braxton Mandara
Canonical implementation	https://github.com/DeKUT-DSAIL/Jasinahub ; Git with the release lockfile, migrations, generated types, and deployment configuration
Licence and languages	MIT; TypeScript, SQL, HTML, and CSS
Baseline	Node.js ≥ 18 , React 18, Vite 5, and the Supabase JavaScript SDK
Developer documentation	https://github.com/DeKUT-DSAIL/Jasinahub/blob/master/JasinaHub%20Developer%20documentation.pdf
Privacy policy and support	https://jasinahub.vercel.app/privacy-policy ; dsail-info@dkut.ac.ke

Source hierarchy. Version-controlled migrations, generated types, configuration, and tested application behaviour are authoritative for implementation. This guide explains their design and development contract, while Section 16 is authoritative for pilot methodology and statistics. Discrepancies are documentation defects and should be corrected with the implementation change that introduced them.

Requirement and evidence language. *Must*, *should*, and *may* denote mandatory, recommended, and optional requirements. *Documented behaviour* reports the release; *required control* states what a deployment must enforce; and *verification status* records supporting evidence or an unresolved check. The application checkout and deployment evidence are maintained externally, so release-specific claims must be tied to an immutable source revision and test result through Section 18.3.

Reading paths. New contributors should begin with the quick start and architecture; backend and security reviewers should continue through roles, database contracts, concurrency, storage, and security; researchers should focus on quality assurance, pilot results, impact, and limitations.

The handbook, companion publication, privacy policy, consent materials, and implementation must use the same pilot facts, workflow rules, model boundary, and governance commitments. Each release records the approved policy and consent versions; illustrative retention periods do not become operational settings until approved and implemented.

2. Introduction

More than 2,000 indigenous languages are spoken across Africa [11], yet most remain underrepresented in speech technology [12]. Data scarcity, linguistic and acoustic variability, constrained computing resources, and ethical concerns continue to impede ASR development [17].

JasinaHub addresses the collection bottleneck through a unified and configurable workflow for spontaneous and read-speech data collection in low-resource settings. Volunteers record voice responses to supplied prompts, approved transcribers produce corresponding verbatim text, and administrators review audio and transcripts before governed export. The backend coordinates authentication, authorisation, task reservation, storage, and recovery.

This guide documents the architecture supporting that workflow. Section 3 defines the software boundary, and Section 16 reports the operational pilot and its limitations.

3. System Scope, Assumptions, and Objectives

JasinaHub supports community-driven collection of speech and corresponding text through modern browsers that support audio input. The release covers prompted recording, transcription, administrative review, and governed dataset export. Training, evaluating, and deploying downstream ASR models are outside the software boundary; the separate *Jasina* project uses governed exports to develop a Kikuyu ASR model.

Collection activities may combine structured spontaneous responses and provided read-speech sentences. New languages and domains can reuse the workflow when their metadata and interaction requirements

fit the existing model, but consent, orthography, transcription guidance, and cultural requirements still require local review.

The design assumes voluntary participation, intermittent connectivity, resource-constrained mobile devices, variable audio quality, and a smaller approved transcriber group. Authentication uses email and password, contributors may skip prompts or leave without submitting, and the workflow does not collect personal clinical records.

The primary objectives are to:

- collect spontaneous and read speech through a mobile-responsive interface;
- prevent conflicting transcription assignments while preserving contributor and transcriber provenance;
- recover bounded local work after interrupted uploads without claiming fully offline operation;
- enforce authorisation, concurrency, and integrity-sensitive rules at the backend;
- retire prompts at a configurable unique-response threshold while measuring speaker diversity separately; and
- export reviewed speech-text pairs with pseudonymized user tags without contributor names.

These are design objectives rather than measured performance claims. Pilot evidence and limitations appear in Sections 16 and 17.1.

4. Developer Quick Start and Reproducibility

The release repository is authoritative for exact versions, migrations, and deployment settings. A supported checkout requires Git, the repository-selected Node.js runtime (at least version 18), the package manager identified by the single committed lockfile, the Supabase CLI, a compatible container runtime, and a microphone-capable browser. Production credentials must never be inferred from examples or committed to source control.

Listing 1: Reference clean-checkout installation workflow.

```
1 git clone https://github.com/DeKUT-DSAIL/Jasinahub.git jasinahub
2 cd jasinahub
3 git checkout v1.0
4 npm ci # use the committed lockfile
```

The repository should contain `package.json` and one lockfile, `.env.example`, Supabase configuration and migrations, deterministic non-sensitive fixtures, versioned Edge Functions, frontend source, automated tests, a changelog, and a short README linking to this guide. Exact runtime and tool versions must be pinned; an unspecified “latest” version is not reproducible.

Copy the environment template to `.env.local`. Browser variables may include the public Supabase URL and anonymous key, but service-role keys, database passwords, SMTP credentials, and cloud-provider private keys remain server-side. Each template entry should state whether it is required, its scope, a non-secret example, and the feature affected when absent.

Listing 2: Reference local backend workflow.

```
1 cp .env.example .env.local
2 npx supabase start
3 npx supabase db reset
4 npx supabase status
```

The reset must apply every migration and load deterministic fixtures into the local database only. Serve required Edge Functions with the repository’s documented command and record the local API URL and anonymous key in `.env.local`.

Listing 3: Reference frontend verification workflow.

```
1 npm run typecheck
2 npm run lint
3 npm test
4 npm run dev
```

Seed data should provide disposable identities for each permission class. Production administrator creation is an audited bootstrap or existing-administrator action, never a public signup option.

A clean-checkout release is accepted only when it can install from the frozen lockfile, initialise an empty local backend, load fixtures, validate generated types, pass unit/database/RLS/concurrency/end-to-end tests, build the frontend, and complete a test recording-to-export cycle without undocumented intervention. Microphone tests require HTTPS or localhost.

5. Related Work and Literature Review

African ASR remains constrained by limited language coverage, insufficient labelled data, acoustic variability, infrastructure limitations, and governance requirements [12, 17]. Mozilla Common Voice demonstrates browser-based crowdsourcing and community validation of read speech [3], while Masakhane shows the value of participatory African-language research, although its cited work concerns machine translation rather than speech collection [20]. Earlier projects such as Lwazi and GlobalPhone established the value of multilingual speech resources [7, 25].

Mobile tools provide closer operational comparisons. Aikuma and LIG-Aikuma support recording, respeaking, translation, and elicitation for language documentation [8, 14]; ALFFA demonstrates downstream ASR use of Sub-Saharan African corpora [13]; and the Makerere Radio Speech Corpus provides 155 hours of Luganda broadcast speech with baseline ASR results [19].

Recent East African resources demonstrate both progress and continuing need. KenCorpus combines text and speech for Swahili, Dholuo, and Luhya [28]; AfriVoices-KE reports approximately 3,000 hours of scripted and spontaneous speech from 4,777 speakers across five languages [29]; and Sunflower shows how locally assembled corpora can support downstream language technologies [1]. Kikuyu ASR error analysis attributes 38.6% of high-error cases to reference-transcription noise, underscoring the importance of annotation quality as well as corpus scale [2].

Table 2: Comparison with related speech-collection platforms and workflows. A dash denotes a capability not reported in the cited publication, not evidence that it is absent from later software.

Platform	Collection and access	Annotation and quality control	Resilience, governance, and export
Common Voice [3]	Prompted read speech through browser crowdsourcing	Community recording validation	Public multilingual corpus releases; other compared capabilities not reported
Aikuma [8]	Recording, respeaking, and oral translation through an Android app	Collaborative, time-aligned oral interpretation	Compared resilience and export capabilities not reported
LIG-Aikuma [14]	Recording, respeaking, translation, and elicitation on mobile devices	Parallel, time-aligned speech collection	Compared resilience and export capabilities not reported
AfriVoices-KE [29]	Scripted and spontaneous speech through a customised mobile app	Automated and human quality checks	Curated multilingual dataset; compared concurrency controls not reported
JasinaHub	Structured spontaneous and read speech through a responsive PWA	Approved transcription, atomic candidate locking, and administrator review	Interrupted-upload recovery, consent-based collection, and pseudonymised controlled export

The comparison systems emphasise crowdsourced read speech, language-documentation workflows, or dataset production at scale. JasinaHub instead integrates structured spontaneous and read-speech collection, approved transcription, atomic task reservation, administrative review, interrupted-upload recovery, and controlled pseudonymised export. Pseudonymisation reduces direct identity exposure but does not make speech anonymous [26].

Large multilingual corpora and pretrained systems broaden coverage but do not remove the need for reviewed target-language speech-transcription pairs for adaptation and evaluation [5, 6, 9, 21, 22]. JasinaHub therefore complements existing resources; the Kikuyu pilot evaluates the integrated collection workflow rather than corpus scale or ASR performance.

6. System Architecture and Workflows

JasinaHub separates browser interaction from managed backend services and persistent storage. A React 18/TypeScript Progressive Web Application, bundled with Vite, communicates with Supabase

services providing GoTrue authentication, PostgreSQL, object storage, Realtime channels, and Deno Edge Functions. PostgreSQL constraints, RLS policies, triggers, and transactional RPCs enforce integrity-sensitive rules; client validation provides earlier feedback but is not a security boundary. This managed-compute model follows serverless principles [16].

The client uses `MediaRecorder` for WebM/Opus capture, `localStorage` through `recordingStorage` for bounded recovery, a Service Worker for the PWA shell, and the Web Vibration API for optional haptic feedback. IndexedDB is a future option for larger or multi-item recovery queues.

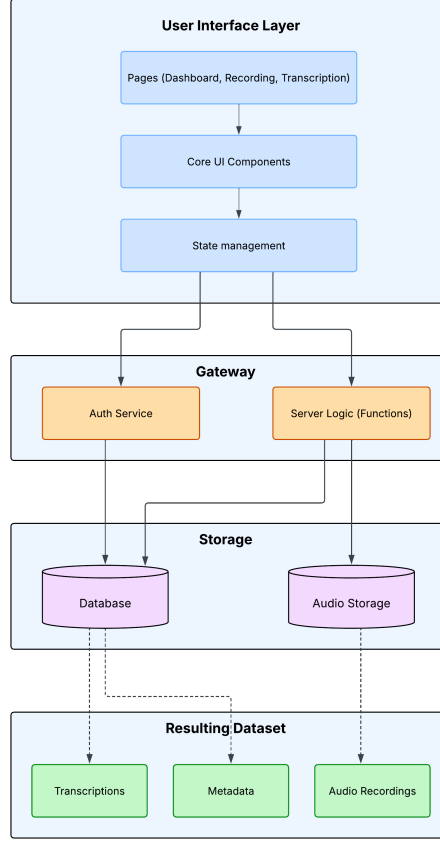


Figure 1: Logical interaction between the JasinaHub client, managed backend services, persistent storage, and exported dataset.

Table 3: Primary workflow contracts.

Workflow	Lifecycle	Authoritative detail
Speech contribution	Verify identity and consent; present a prompt; record, playback, recover if interrupted, upload, and submit for review.	Sections 11 and 11.2.
Transcription	Verify identity and approve the transcriber; atomically reserve accepted audio; listen, draft, submit, and release or expire the lease.	Section 10 and Section 15.2.
Administration	Manage accounts and prompts; review recordings and transcripts; monitor activity; and export eligible pairs.	Sections 14 and 15.

Backend transitions govern persisted workflow states, while recording recovery also includes local browser state before confirmation. Figure 2 summarises the end-to-end path; subsequent chapters define the enforceable states, failure handling, and evidence requirements.

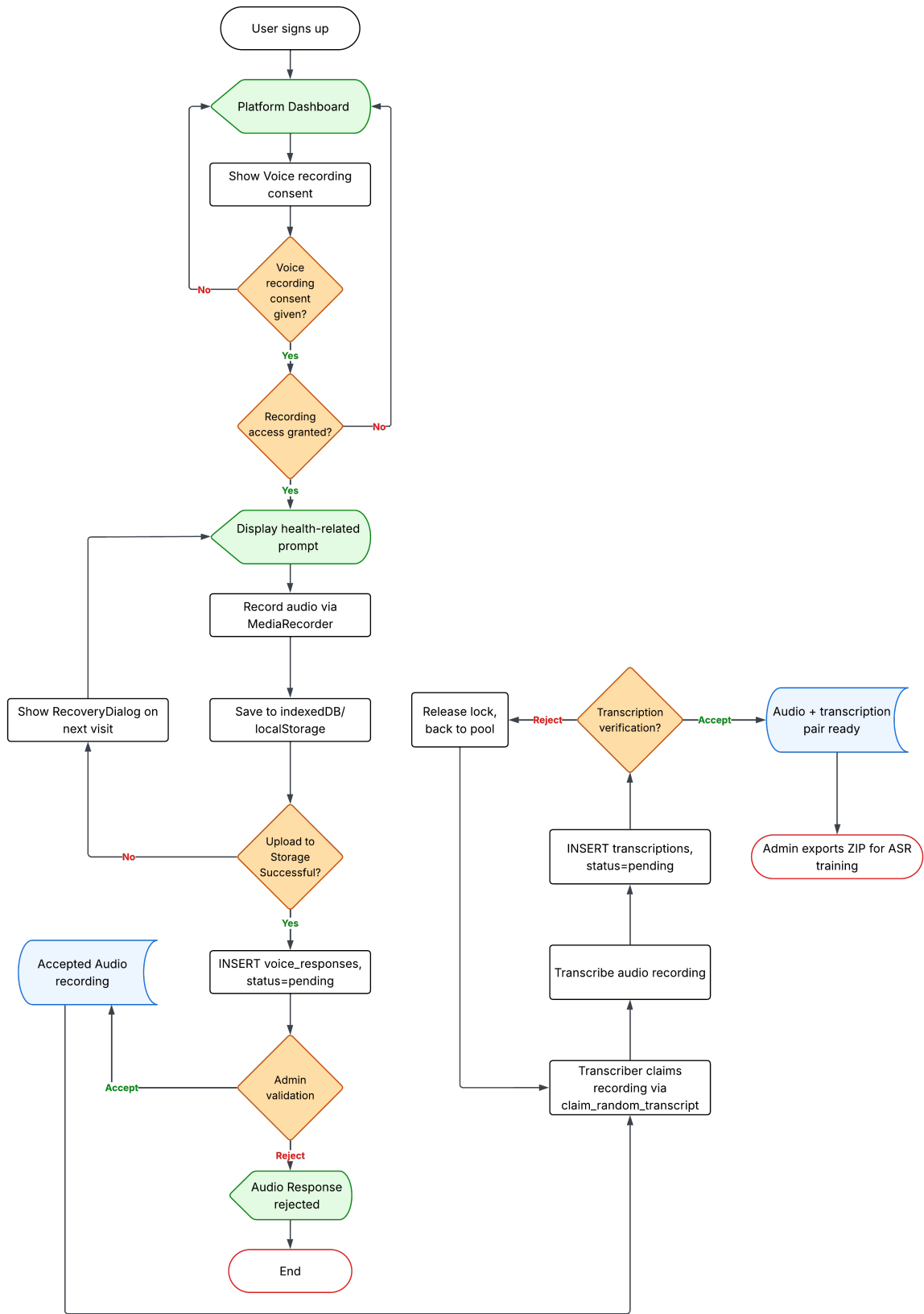


Figure 2: End-to-end recording, moderation, transcription, validation, and export workflow.

7. System Roles and Authorisation Model

JasinaHub stores the PostgreSQL `app_role` values `admin` and `user` in a dedicated `user_roles` table—never on the editable `profiles` table—to reduce profile-update privilege-escalation risk. This separation follows established role-based access-control principles [24]. Role checks are performed by a `has_role(uuid, app_role)` function declared as `SECURITY DEFINER` so that RLS policies can invoke it without recursive policy evaluation. In application terms, the `transcription_approved` profile flag yields three effective operating groups: volunteer, approved transcriber, and administrator.

7.1. Volunteer (Recording Contributor)

The default role assigned at signup. Volunteers must:

- Complete a four-step signup wizard capturing first name, last name, email, password (with strength indicator), phone number, gender, age, and dialect.
- Be marked `verified = true` by an administrator before accessing collection prompts (enforced by RLS on `questions` and `voice_responses`).
- Grant explicit, persistent voice-recording consent (stored as `voice_recording_consent` and `consent_timestamp` on the profile) before recording.
- Optionally update their phone number exactly once via the one-time `phone_number_updated` flag.

7.2. Transcriber

A volunteer who additionally requests transcription work. Transcribers are gated by the boolean `transcription_approved` flag on the profile, which only an administrator may set. They must accept verbatim transcription guidelines (recorded as `transcription_guidelines_agreed` and `transcription_guidelines_agreed_at`) before claiming any task.

7.3. Administrator

A role assigned by inserting an `admin` entry in `user_roles`. Administrators have full visibility through the `has_role(auth.uid(), 'admin')` predicate that appears in nearly every RLS policy. They manage prompts, verify users, approve transcribers, moderate responses, validate transcriptions, export datasets, and access observability tooling.

7.4. Authorization Matrix

Table 4 separates stored roles from account states. Every cell must be backed by an RLS policy, storage policy, or controlled function test; hiding a button is not authorization.

Table 4: Minimum authorization matrix.

Operation	Anonymous	Unverified user	Verified volunteer / approved transcriber	Administrator
Read active prompts	No	No	Yes	Yes
Insert own recording	No	No	Yes	Policy-defined
Read own recording	No	No	Yes	Yes
Moderate recordings	No	No	No	Yes
Claim transcription	No	No	Approved transcriber only	Policy-defined
Edit own unaccepted transcription	No	No	Original transcriber only	Yes
Validate transcription	No	No	No	Yes
Manage roles and approvals	No	No	No	Yes
Read feedback aggregates	No	No	No	Yes
Export accepted dataset	No	No	No	Yes

Policy-defined administrator behaviour must be stated explicitly in the implementation reference: an administrator should not implicitly submit research data merely because the account can bypass ordinary restrictions. Revoking verification or transcription approval must define what happens to in-progress uploads, existing data, and active locks.

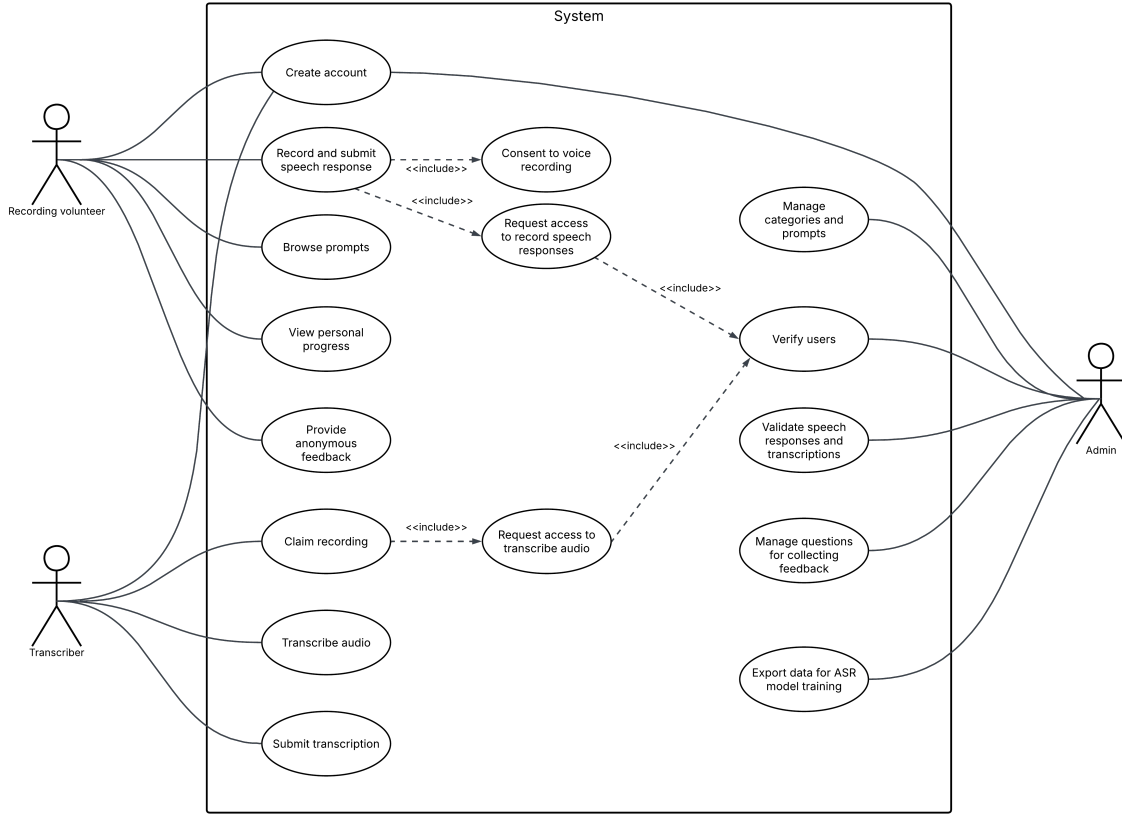


Figure 3: Use case diagram showing the three actors (Volunteer, Transcriber, Administrator) and the fourteen primary use cases of the JasinaHub system.

8. Development and Contribution Approach

JasinaHub was developed through iterative, AI-assisted prototyping under human review [10]. Lovable scaffolded the initial authentication, recording, transcription, and administration workflows. Google Antigravity later supported component development and refactoring, including stricter TypeScript typing, transactional transcription reservation, interrupted-upload recovery, PWA support, curation tools, and an embedded test runner.

AI assistance did not replace developer oversight. Application logic and user workflows were manually verified, while security-sensitive RPCs, RLS policies, schema migrations, and database configuration were reviewed and tested before deployment. All changes remain subject to source control, type checking, migration review, automated tests, and the release checks in Section 18.3.

The architecture places authorisation and contested task assignment in PostgreSQL and specialised work in managed Edge Functions. These choices reduce client-side coordination but do not establish a tested capacity limit. New languages and additional domains may also require new metadata, scripts, moderation rules, or consent controls rather than prompt configuration alone.

The repository should separate routes and presentation components from recording, transcription, administration, shared services, offline persistence, generated database types, migrations, Edge Functions, and tests. Page components must not duplicate RLS-sensitive eligibility logic or issue ad hoc privileged queries.

A change affecting schema, authorisation, workflow state, configuration, or a public data contract must update the migration, generated types, automated tests, documentation, and changelog together. Applied migrations are immutable; corrections use forward migrations. Review requires formatting, linting, type

checking, tests, and a production build.

9. Database Design and Domain Invariants

Release v1.0 uses a normalised PostgreSQL schema with eleven application tables and the `voice-recordings` object-storage bucket. Migrations remain authoritative for column types, defaults, keys, deletion behaviour, indexes, triggers, grants, and RLS policies.

Table 5: JasinaHub public-schema overview.

Table	Purpose	Representative fields
<code>profiles</code>	User metadata and workflow state	dialect, age, verification, consent, transcription approval
<code>categories</code>	Prompt groupings	<code>name</code> , <code>description</code>
<code>questions</code>	Ordered spontaneous or read-speech prompts	<code>question_text</code> , <code>category_id</code> , <code>order_index</code>
<code>voice_responses</code>	Audio submissions and moderation state	object locator, duration, status, prompt and contributor IDs
<code>transcriptions</code>	Canonical verbatim text	response and transcriber IDs, text, edit count, status
<code>transcription_locks</code>	Exclusive configurable leases	response and transcriber IDs, lock and expiry times
<code>user_progress</code>	Per-user category progress	user/category IDs, completed and total questions
<code>user_roles</code>	Privileged roles separate from profiles	<code>user_id</code> , <code>app_role</code>
<code>user_activity_logs</code>	Administrative audit stream	actor, action, page, JSONB details
<code>feedback_questions</code>	Configurable feedback prompts	type, required/active flags, order
<code>feedback_submissions</code>	Feedback without direct account identifiers	question, answer, submission time

Object storage: `voice-recordings` stores uploaded WebM/Opus objects.

Table 6: Core release invariants and enforcement requirements.

Rule	Authoritative enforcement
Configured recording limit; 30 seconds in the pilot	Validate trusted media duration or inspect the uploaded object; a caller-supplied number alone is insufficient.
One response per contributor and prompt	Unique backend constraint on (<code>user_id</code> , <code>question_id</code>) and concurrent-submission tests.
Prompt retirement; three unique respondents in the pilot	Re-evaluate the configured threshold transactionally during submission and record its effective date.
One canonical transcription per recording	Unique response reference plus a controlled submission path that verifies an active lock.
Configurable correction allowance; three in the pilot	Backend constraint, trigger, or controlled RPC; client counters are advisory.
Original-transcriber edit rights	Owner-and-status RLS for ordinary edits, with a separate audited administrator path.
Valid workflow states and feedback ranges	Database enums/checks or controlled RPCs, not TypeScript-only validation.

Category progress must derive from canonical contribution records. Transcription remains verbatim under the conventions in Section 15.2; prompt retirement broadens coverage but does not by itself establish speaker diversity.

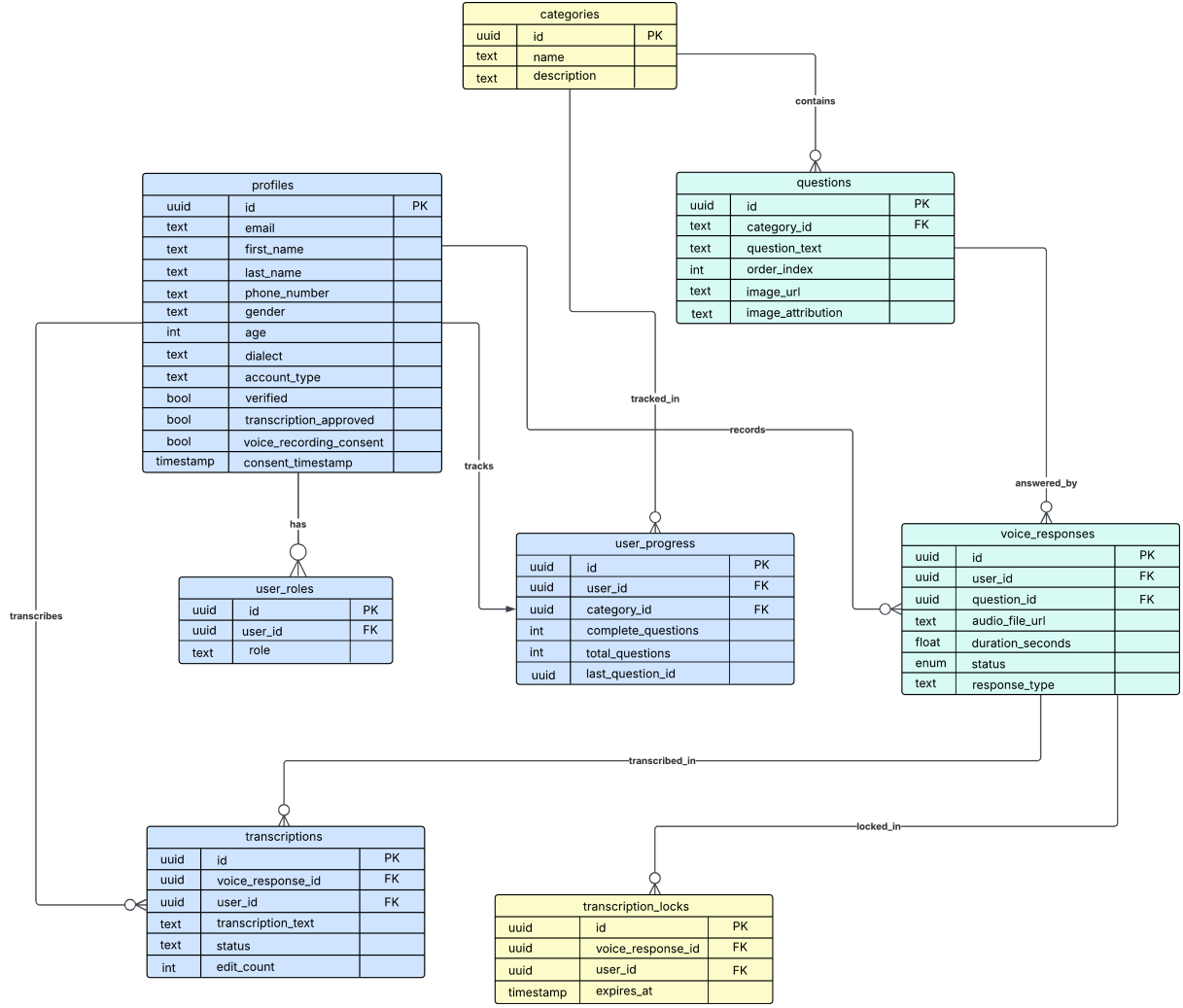


Figure 4: Conceptual entity relationships for the core recording and transcription workflow; release migrations remain authoritative.

Each schema change uses a timestamped forward migration tested against an empty database and a production-like snapshot. Seed data must be synthetic. Before destructive production changes, operators verify recoverable database and storage backups and document forward recovery.

RLS and privileged-function tests must exercise anonymous, ordinary, verified, approved-transcriber, administrator, and service-role contexts. Every **SECURITY DEFINER** function requires a trusted owner, fixed safe **search_path**, minimal grants, caller validation through **auth.uid()**, and explicit tests against cross-user access.

10. Concurrency and Consistency Management

The **claim_random_transcription** RPC selects an eligible accepted recording and creates an exclusive lease in one PostgreSQL transaction, preventing concurrent assignment of the same candidate [15]. The pilot used a configurable ten-minute lease.

The function must derive the claimant from **auth.uid()**, verify transcription approval and guideline acceptance, exclude recordings with a canonical transcription or active lock, and use a row-locking strategy such as **FOR UPDATE SKIP LOCKED** with a unique response constraint. A caller with an existing lock must receive the same task or a defined error; deployments must also define randomisation, skip-exclusion duration, and starvation handling.

Table 7: Transcription lease lifecycle.

State	Entry	Exit
Available	Accepted recording without a transcription or active lease	Atomic claim
Claimed	Authenticated approved transcriber owns an unexpired lease	Submit, skip, logout, administrator action, or expiry
Expired	Database time reaches <code>expires_at</code>	Reclaim or cleanup
Transcribed	Canonical text is stored	Audited administrator reopen only

Submission must occur through one controlled transaction that verifies a still-valid caller-owned lease, inserts or updates the canonical transcription, and releases the lease. `release_transcription_lock` is owner-scoped and idempotent. Skipped candidates may be temporarily excluded for that transcriber without becoming globally ineligible.

Concurrency tests run parallel database sessions and prove exclusive claims, owner-only release, expiry and reclaim behaviour, user-scoped skip exclusion, and idempotent retry after ambiguous network failures. Tests use the configured lease rather than assuming ten minutes globally.

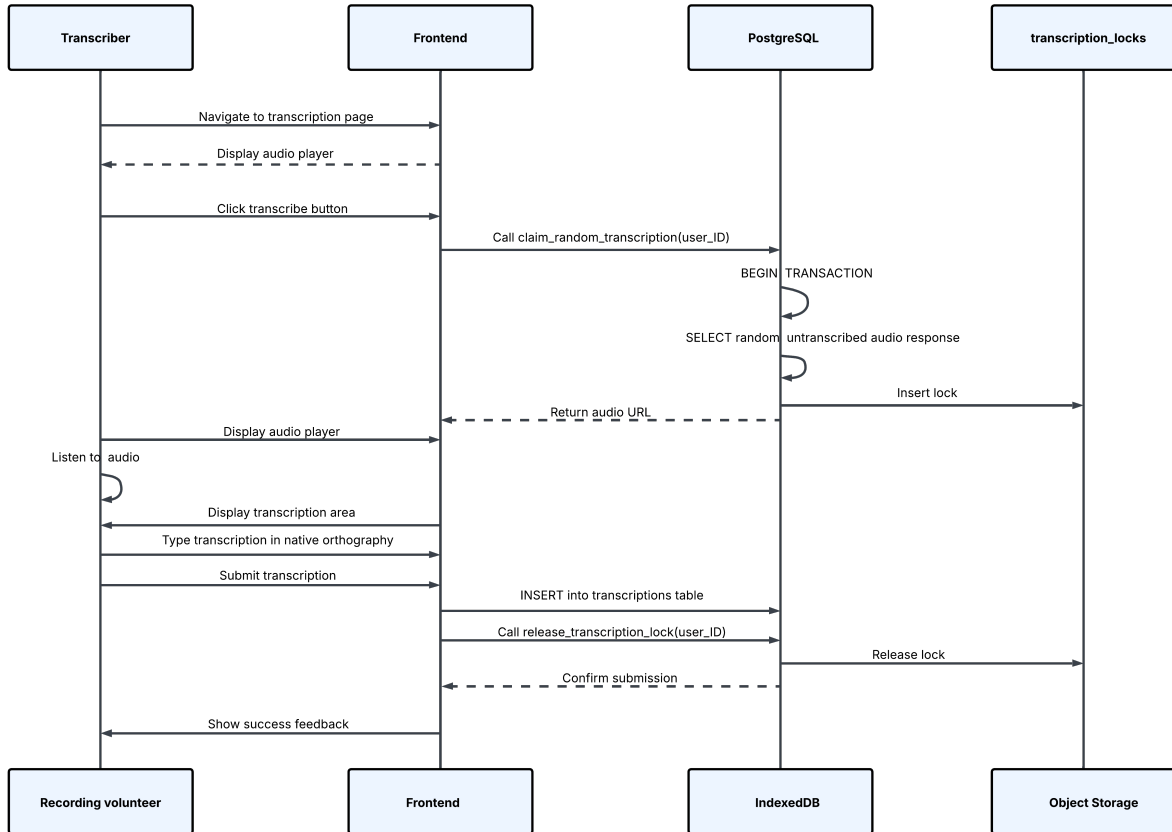


Figure 5: Atomic transcription claim and submission workflow.

PostgreSQL can make a critical database transition atomic, but browser persistence and object storage cannot join that transaction. Recording submission therefore follows an idempotent saga with reconciliation for orphaned objects or incomplete rows.

11. Voice Recording and Recovery

The browser `MediaRecorder` API captures WebM/Opus audio for structured spontaneous prompts or predetermined read-speech sentences [27, 30]. Desktop capture uses `AudioWaveform`; the compact mobile interface uses `MinimalRecorder`. Volunteers may skip unfamiliar prompts and replay a recording before submission.

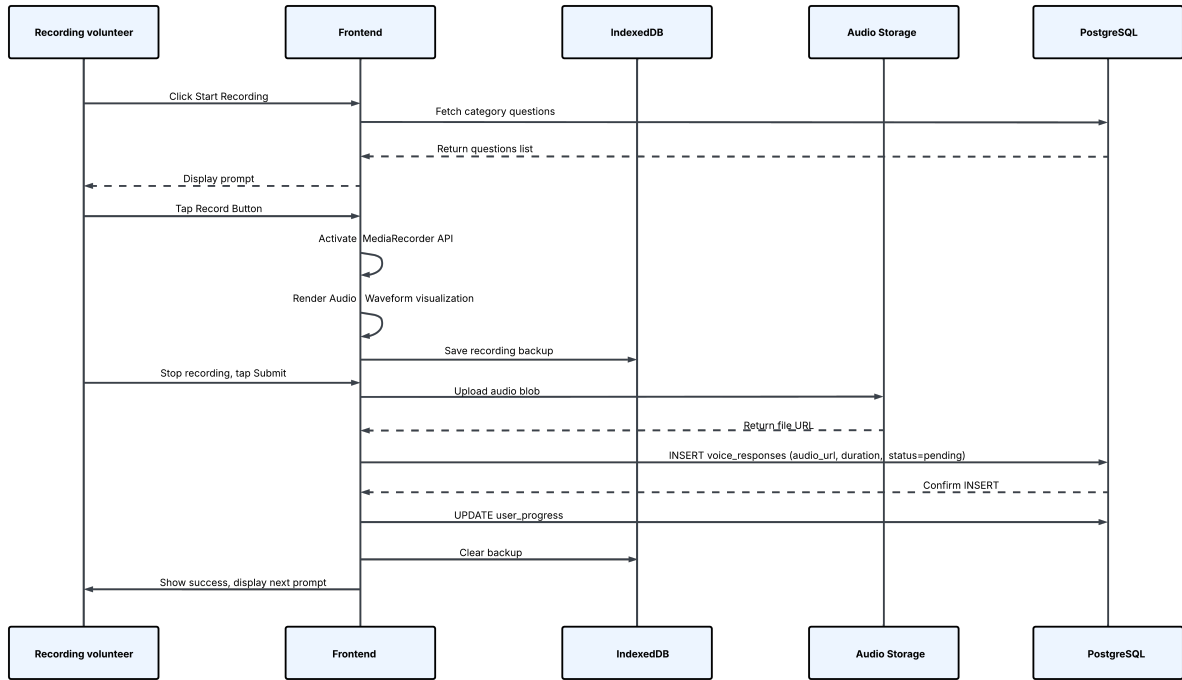


Figure 6: Recording, local recovery, object upload, and database-persistence sequence.

Submission saves a bounded recovery copy, uploads the audio to the private `voice-recordings` bucket, inserts a pending `voice_responses` row, updates progress, and clears the local copy only after the object and row are durably confirmed. A stable client-generated submission identifier makes retries idempotent.

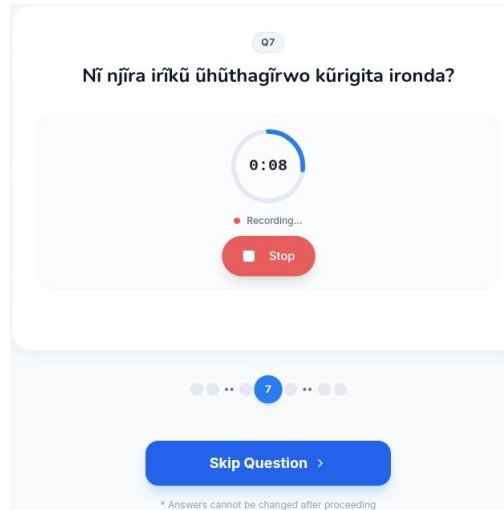


Figure 7: Mobile recording interface showing the active prompt, timer, progress, and skip control.

11.1. Audio Contract

The deployment records the actual MIME type, sample rate, channel count, bitrate or quality setting, maximum byte size, duration limit and tolerance, and fallback behaviour. The pilot used a 30-second limit. Unsupported formats must be rejected with actionable guidance or explicitly preserved in the export manifest. The database stores a canonical bucket and object path; authorised readers receive short-lived signed URLs. Checksums should detect corruption where feasible.

11.2. Interrupted-Upload Recovery

Before upload, `recordingStorage` serialises one bounded Base64 recording in `localStorage`. `RecoveryDialog` offers retry on the next visit and `OfflineIndicator` reports network state. This mechanism does not provide fully offline authentication, transcription, or submission: `localStorage` is synchronous, space-inefficient for binary data, quota-limited, and subject to eviction. IndexedDB is recommended before supporting multiple queued recordings.

The recovery record contains the submission, owner, prompt, payload, MIME type, duration, creation time, retry count, and last error. It moves through `local`, `uploading`, `confirmed`, or `failed` states. Logout must not submit retained audio under another identity, and users must be able to inspect and delete abandoned work.

Table 8: Recording and recovery failure handling.

Failure	Required behaviour	Recovery
Permission denied	Store no unusable placeholder	Show browser-specific guidance and retry
Network loss	Retain the bounded recovery record	Resume with the same submission identifier
Upload succeeds; row insert fails	Do not announce completion	Retry the insert or reconcile the orphan
Row exists; response is lost	Query by submission identifier	Treat the matching row as success
Quota exceeded	Stop safely and explain the limit	Free storage or submit while online

11.3. Worked Lifecycle

A verified, consenting volunteer records and previews a response. If connectivity fails, the next session retries the retained copy without duplicating the submission. An administrator accepts usable audio, an approved transcriber claims and submits verbatim text under a valid lease, and an administrator validates the pair for governed export. Each recovery path preserves ownership, consent, review, and export checks.

12. Backend Interface Reference

This section summarises documented RPC purposes, intended callers, and required failure handling. It is not a copy-ready API specification: exact parameter names, return schemas, and deployed grants have not been verified against an application checkout. Use the tagged SQL definitions and generated TypeScript types for call signatures. Section 18.3 defines the evidence needed to complete the release reference.

12.1. Core RPC Contracts

Table 9: Documented RPC behaviour and required failure handling; exact signatures require release verification.

RPC	Intended caller	Documented purpose	Required failure behavior
<code>claim_random_transcription</code>	Authenticated approved transcriber	Reserves an eligible accepted response and returns recording metadata with a configured lease	Not authenticated, not approved, no work, or conflicting state; no partial lock
<code>release_transcription_lock</code>	Authenticated lock owner	Releases the caller's claim on a documented skip, logout, submission, or abandonment path	Idempotent if already released; cannot release another user's lock
<code>get_question_counts</code>	Verified user or administrator	Counts used for progress and prompt availability	Stable empty result; must not expose restricted prompts
<code>get_question_unique_user_counts</code>	Verified user or administrator	Unique respondent count per permitted question	Same filtering and threshold semantics as submission path
<code>get_admin_chart_data</code>	Administrator only	Aggregate operational data with a documented time zone and interval	Authorization failure without leaking aggregates

Error-handling requirement. Do not expose raw PostgreSQL error text as a stable client API. Map expected failures to stable application codes such as `TRANSCRIBER_DENIED`, `NO_AVAILABLE_WORK`, `LOCK_EXPIRED`, and `TRANSCRIPTION_CONFLICT`. These are documentation-level codes, not verified wire responses. Unexpected failures should carry an internal correlation identifier.

12.2. Edge Function Contracts

The documented `reset-password` function supports password recovery. The `migrate-drive-audio` and `upload-to-gcs` functions belong to legacy storage operations rather than ordinary contributor workflows. Their exact HTTP contracts and current deployment status are not established here.

Access requirement. Migration functions must be restricted to authorised administrative use. Legacy functions should be disabled or removed once their retention requirement ends. Before invoking any of these functions, verify its current authentication, request schema, and operational limits using the release reference.

13. Storage and Data Lifecycle

Canonical audio resides in the private Supabase `voice-recordings` bucket. Database rows store stable object locators rather than expiring signed URLs; authorised playback generates short-lived access. Bucket policy must define upload owners, readable paths, MIME and size limits, signed-URL lifetime, and deletion behaviour. Object names use opaque identifiers rather than emails or names.

The release also documents a password-reset Edge Function and legacy Google Drive/Google Cloud migration tools. Legacy functions require administrator-only access and should be disabled when no longer needed. Prompt images require an explicit hosting and authorisation model rather than an assumed public bucket.

Database and storage lifecycles must remain linked. Reconciliation reports rows without objects, objects without rows, duplicate submission identifiers, inaccessible objects, and checksum failures. Consent withdrawal and deletion procedures must locate both the database records and corresponding audio.

The published policy names Supabase and Vercel, but each deployment must record its actual hosting, storage, backup, and recipient locations and approve any international-transfer safeguards. Legacy Drive or Google Cloud paths require the same inventory while retained.

Retention settings are governed by approved purpose, consent, protocol, and erasure requirements. The schedule covers accounts, consent and rights requests, rejected and accepted contributions, activity and security logs, local recovery copies, backups, exports, and downstream research artefacts. For each category it records the trigger, period, owner, deletion or de-identification method, exceptions, and verification evidence.

Migrations must use a stable source identifier or checksum, preserve a machine-readable manifest, verify counts and playback, and retain the source until the approved verification period ends. Backup, restore, deletion, and incident procedures must be documented and tested for each deployment before production data are migrated.

14. Security, Privacy, and Consent

JasinaHub places table authorisation in PostgreSQL grants and RLS after the authenticated request establishes an identity and claims context. Object storage, Edge Functions, Realtime, and privileged service operations have separate controls. Privileged roles reside in `user_roles`, while transcription approval remains a controlled profile capability; this separation reduces profile-update privilege-escalation risk. The `has_role()` `SECURITY DEFINER` function requires a trusted owner, fixed safe `search_path`, narrow grants, and dedicated tests.

GoTrue provides email-and-password authentication; third-party OAuth providers are not enabled. Administrator verification gates protected collection workflows, and approved transcribers must also accept the transcription guidelines. Password recovery uses a dedicated Edge Function with approved redirect origins and non-enumerating responses.

The threat model assumes modified clients, stolen ordinary credentials, malicious uploads, replay and concurrency, leaked signed URLs, and accidental administrator actions. Security review covers:

- RLS and storage-policy bypass for every role and account state;
- cross-user ownership changes and insecure direct-object access;

- privileged-function ownership, grants, caller identity, and auditability;
- upload validation, object-name handling, secrets, session policy, and recovery; and
- dependency, migration, destructive-export, and administrator-action controls.

14.1. Privacy and Consent

The published policy identifies Dedan Kimathi University of Technology, acting through DSAIL, as controller for the hosted service. Requests go to dsail-info@dkut.ac.ke; another deployment must identify its own controller and notice.

Before recording, volunteers receive the applicable policy and consent form. Capture begins only after `voice_recording_consent` and `consent_timestamp` are recorded, with the accepted versions preserved. Consent explains participation, cloud storage, compatible research use, controlled sharing, pseudonymisation, residual voice-identification risk, and the rights to decline, stop, withdraw, access, correct, close an account, and request eligible erasure. Suspected under-age participation is escalated for review and removal where required.

The platform processes account and contact data, demographic and dialect metadata, workflow states, contributions, consent records, feedback, and linked operational records. Registration distinguishes required from optional fields; passwords, tokens, and signed audio URLs are excluded from application logs and exports. Future use remains limited to the compatible speech, language, accessibility, and data-quality research described at consent, with fresh approval or consent where materially different use requires it.

Supabase and Vercel support the hosted service; authorised staff, approved transcribers, governed research recipients, institutional oversight personnel, and lawful authorities may receive access only for approved purposes. Open-source software access does not grant participant-data access. Names are removed from governed exports, but voice, metadata, and response content may still identify a speaker [26].

Each deployment must document how it handles access, correction, restriction, withdrawal, erasure, recipient notification, complaints, backups, and downstream copies. Documentation and technical controls support, but do not by themselves establish, compliance with Kenya’s Data Protection Act [23].

15. Quality Assurance and Dataset Export

Quality control has two stages: administrators first review recordings for transcription eligibility, then compare submitted transcripts with the source audio before a pair becomes export eligible.

15.1. Recording Review

Accepted recordings are intelligible, relevant to the assigned prompt, primarily in the target language, and free of direct identifying content. Read speech is also checked against the displayed sentence without treating natural accent or dialectal realisation as an error. Minor noise, pauses, fillers, laughter, repetitions, or code-switching remain acceptable when the response is usable.

Table 10: Recording rejection criteria.

Reason	Operational interpretation
Excessive noise	Noise, clipping, distortion, or competing speech prevents reliable use.
Silence	The clip contains no meaningful spoken response.
Wrong language	The response is not primarily in the target language; limited natural code-switching is acceptable.
Off-topic response	The response does not address the prompt.
Identifying content	The recording may directly identify the speaker or another person.

15.2. Transcription and Validation

Approved transcribers and reviewers use one verbatim-transcription standard:

- retain fillers such as [mmh], [eeh], and [aah], and mark noticeable pauses as [pause];



Figure 8: Administrator overview of collection statistics and review queues; screenshot values are not authoritative pilot totals.

- mark inaudible or unreliable segments with [?] rather than guessing;
- retain code-switched words inside paired [cs] markers;
- apply ordinary punctuation and capitalisation, writing numbers as spoken;
- mark cut-off words with a trailing hyphen and preserve repetitions;
- retain the speaker’s dialectal form under the agreed orthography; and
- replace direct identifiers in text with [personal information] and flag the source recording.

The transcriber must play the assigned clip to completion before text entry is enabled. Replay and speed adjustment are available; drafts survive connectivity interruptions. Skip, logout, submission, or lease expiry returns unfinished work to the queue. The pilot allowed three post-submission corrections. Shared conventions and review are important because reference-transcription noise is a major contributor to high-error Kikuyu ASR cases [2].

15.3. Dataset Export

Administrators export accepted pairs as audio files plus a versioned CSV manifest and dataset-level metadata. Audio downloads use batching and bounded retry; a metadata-only fallback is explicitly labelled incomplete. The export excludes withdrawn consent, rejected media, unaccepted transcripts, missing objects, and failed checksum validation according to a documented policy.

Contributor names are replaced by stable JSN-XXXXXXX identifiers; the same contributor retains one identifier within the governed namespace. The identity mapping and any generation secret are excluded. Spoken identifiers require audio review or approved redaction because replacing transcript text alone does not remove them from the recording.

The manifest records the audio path, language and dialect, prompt and category identifiers, verbatim transcript, duration, actual MIME type, review state, pseudonymous provenance where permitted, and UTC creation and validation times. Dataset-level metadata records schema and software versions, generation time, record count, total duration, and checksums. Export uses a consistent snapshot or documented cut-off time, and recipient records support later restrictions or erasure notices.

15.4. In-Product Feedback

Authenticated users may answer administrator-configured questions anonymously. Responses omit a direct `user_id`, but timestamps and infrastructure logs can permit correlation; “anonymous” therefore

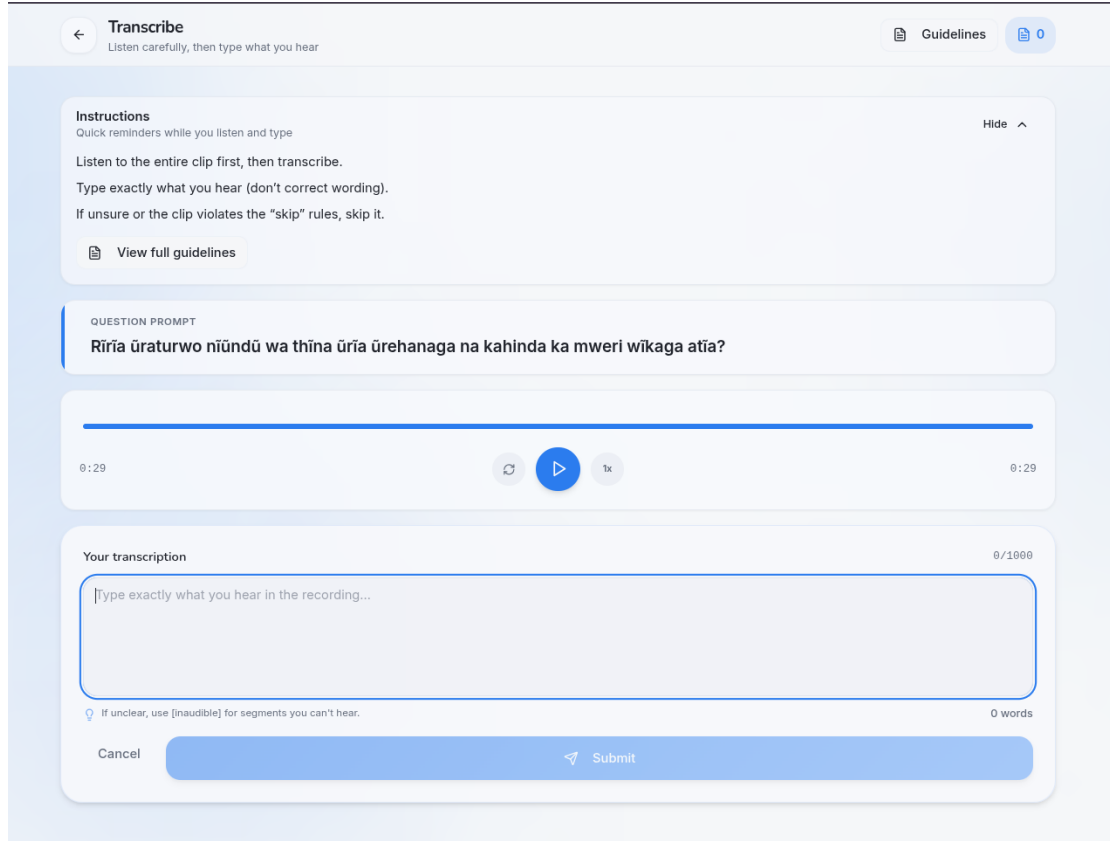


Figure 9: Transcription workspace with guidance, prompt, audio controls, and verbatim text entry.

describes data minimisation rather than a guarantee. Administrators receive aggregate analysis and controlled raw-text access.

16. Pilot Deployment and Dataset Statistics

The pilot assessed the complete corpus-creation workflow—prompted recording, upload, review, transcription, curation, and export—under practical deployment conditions. It was not a formal usability study or an ASR evaluation. Release v1.0 was piloted with 25 adult Kikuyu speakers and 369 health-related prompts. The 2019 census identifies the Kikuyu as Kenya’s largest ethnic community [18], while AfriVoices-KE reports Kikuyu as its largest language subset, with 754 hours from 1,331 contributors [29]. Nevertheless, Jasinahub remained configurable for other languages and domains.

Participation was voluntary and compensated, with eligibility limited to adults (18 years or older) fluent in Kikuyu. The sample covered four regional varieties: *Kĩ-mũrang’a* (Murang’a), *Kabete* (Kiambu), *kĩ-ndia/gĩ-gĩchũgũ* (Kirinyaga), and *kĩ-mathĩra* (Nyeri). Before recording, participants completed an electronic informed-consent process covering the project purpose, recording activities, secure storage, controlled future research use, pseudonymisation, residual voice-identification risk, and the rights to decline, stop, withdraw, access contributions, close an account, and request erasure.

Recording ran from 16 January to 18 February 2026, followed by transcription from 17 March to 14 May 2026. Participants answered multiple prompts, and prompts received responses from multiple speakers. Five participants also served as trained, approved transcribers.

The three-response retirement rule was introduced during collection. Earlier submissions were retained, so some prompts exceeded the threshold and the final total was not limited to 369×3 recordings.

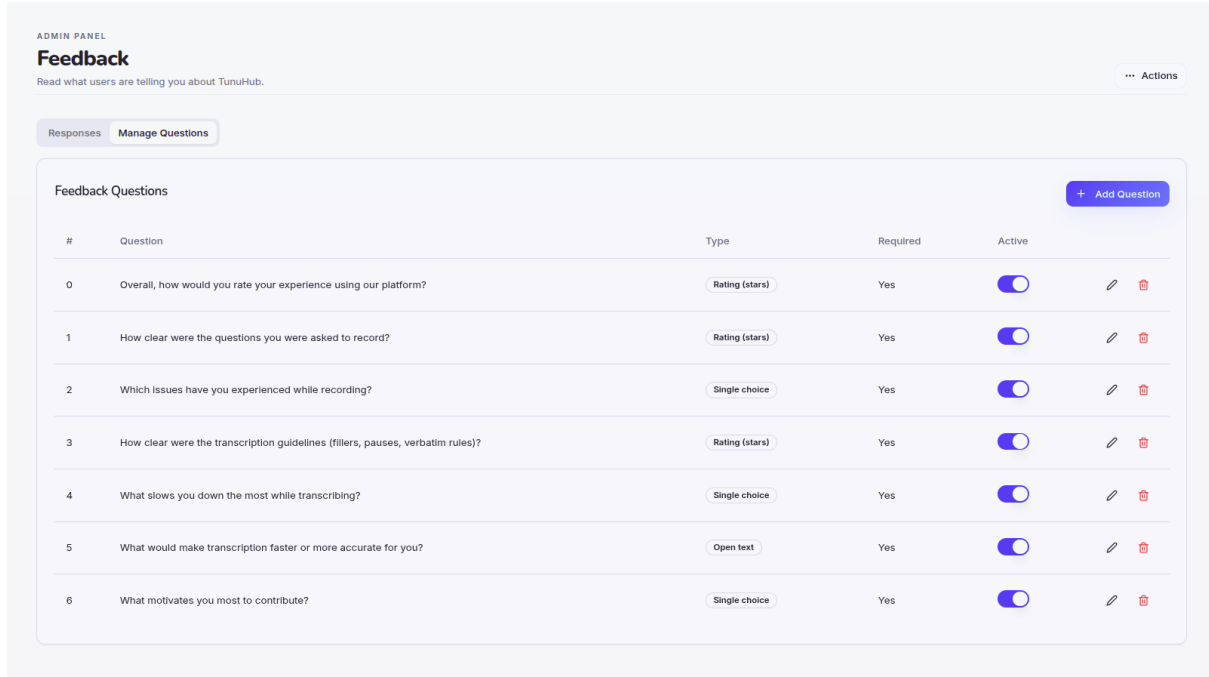


Figure 10: Administrator configuration of feedback questions and response types.

Table 11: JasinaHub pilot deployment and quality-review outcomes.

Pilot metric	Value
Participants	25
Approved transcribers	5
Distinct health-related prompts	369
Recordings submitted	2,275
Total collected duration	16 h 18 min
Accepted recordings	1,789 (78.6%)
Accepted recording duration	13 h 16 min (81.4%)
Completed transcriptions	1,789
Completed speech–transcription pairs	1,789
Excluded recordings	486
Excluded audio duration	3 h 02 min

Administrators reviewed recordings before transcription and then checked completed transcripts against the source audio. The workflow retained 1,789 recordings and excluded 486 under the criteria in Section 15.2. All retained recordings were transcribed, yielding 1,789 completed speech–transcription pairs. Retention was 78.6% by count and 81.4% by duration, indicating that retained clips were longer on average. These results demonstrate operational use of collection, review, transcription, curation, and export; they do not establish usability, ASR quality, population representativeness, transcription agreement, or production-scale capacity. The curated pairs are being used to develop *Jasina*, a Kikuyu ASR model.

Curated export access remained subject to project authorisation procedures, the published privacy policy at <https://jasinahub.vercel.app/privacy-policy>, and safeguards for personal and biometric data under Kenya’s Data Protection Act [23]. Stable JSN-XXXXXXX pseudonyms replaced contributor names while preserving same-speaker linkage. The dataset was not treated as anonymous because speech may retain identifying characteristics [26].

17. Impact and Reuse Potential

JasinaHub’s immediate contribution is an operational workflow that converts community speech contributions into reviewed audio–transcription pairs. The Kikuyu pilot produced 1,789 pairs now being

used to develop *Jasina*, providing a direct research application of the platform’s output. The workflow is consistent with participatory African-language research principles [20].

By consolidating recording, transcription assignment, review, and export, the platform reduces reliance on manual file allocation and separate tracking tools. Configurable categories and prompt sets support reuse across languages and domains, but each deployment must review language metadata, transcription conventions, consent, cultural appropriateness, data residency, moderator capacity, and downstream licensing.

Evidence of impact remains limited to the reported pilot and ongoing *Jasina* development. The platform has not yet been adopted outside the project, and the study reports no commercial use or comparative measurement of time or cost savings.

17.1. Limitations

The pilot tested corpus creation rather than ASR performance or wider effects on digital inclusion; evaluating *Jasina* requires a separate study. Its 25 participants and health-related Kikuyu prompts do not establish usability, speech diversity, or data quality across other languages, communities, or domains. Reviewers used a shared guide, but inter-annotator agreement was not measured. Pseudonymisation reduces direct-identifier exposure but cannot eliminate re-identification risk from voice, demographic metadata, or response content, so dataset access remains controlled.

18. Observability, Testing, and Release Evidence

18.1. Operational Visibility

`useActivityTracker` records significant actions such as page visits, recording submissions, and transcription completion in `user_activity_logs`; Realtime channels expose near-live activity and presence to administrators. Audit records include actor, action, target, outcome, UTC time, environment, and correlation identifier while excluding secrets, contact details, transcript text, tokens, and signed URLs. Because operational records may remain personal data, they belong in the data inventory, rights-request searches, and retention schedule.

Production monitoring should cover authentication failures, client exceptions, upload success and latency, queued-upload age, orphaned objects, RPC errors, lock contention and expiry, moderation backlog, Edge Function failures, database resources, storage growth, and export failures. Each alert needs an owner, severity, threshold, and runbook.

18.2. Test Strategy

The administrator interface includes authenticated smoke suites for sessions, roles, RLS, RPCs, storage, business rules, accessibility, compatibility, resilience, and end-to-end workflows. These checks are useful for deployed verification but do not replace isolated automated tests. Destructive cases use dedicated fixtures and are never run against ordinary production work.

Table 12: Required automated test layers.

Layer	Coverage	Execution
Unit/component	Validation, recorder controls, state, formatting, retry	Pull request
Database/RLS	Constraints, triggers, policies, grants, privileged functions	Migration and pull request
Integration	Client, storage, RPC, authentication, recovery	Isolated local backend
Concurrency	Parallel claims, expiry, release, idempotent retry	Pull request and release candidate
End-to-end	Signup through governed export	Release candidate and staging
Accessibility	Keyboard, focus, names/roles, contrast	Pull request and manual review
Performance/load	Upload, contention, export, dashboard queries	Capacity-changing release
Recovery	Restore, failed migration, orphan and local recovery	Scheduled exercise

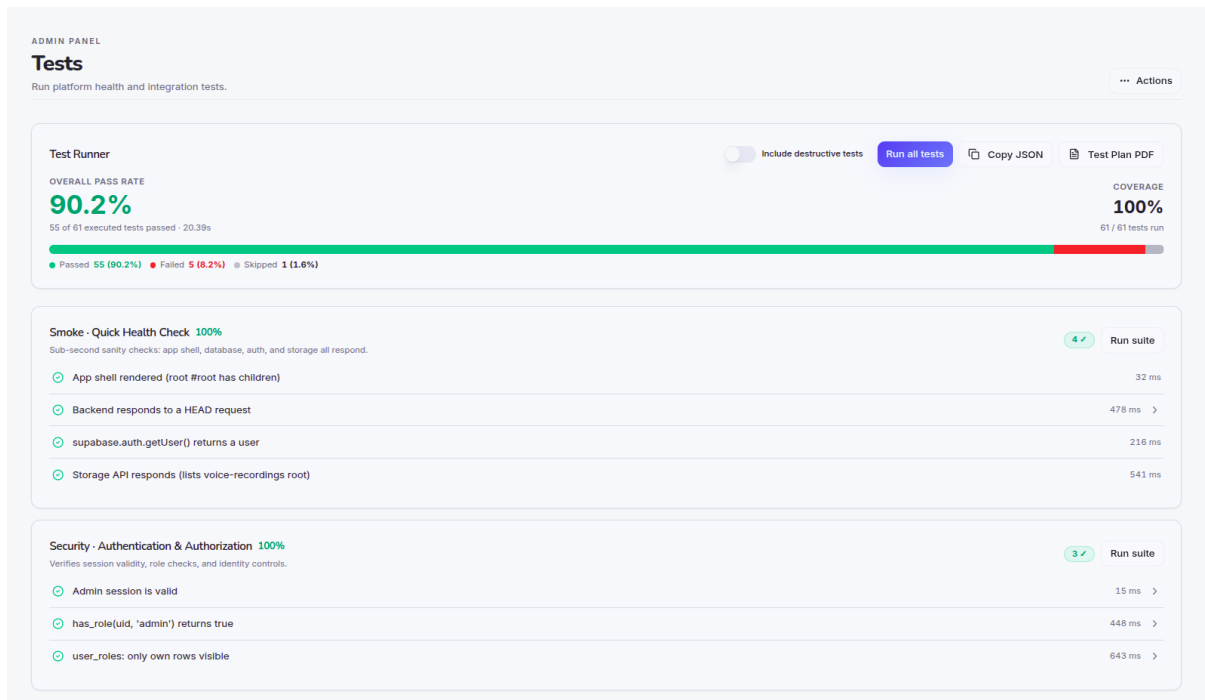


Figure 11: Administrator test runner showing point-in-time suite results.

The repository documents commands, fixtures, mutation behaviour, expected duration, and pass thresholds for every layer. Continuous integration installs from the frozen lockfile, lints, type-checks, builds, initialises an isolated backend, applies migrations, runs tests, and verifies generated database types.

18.3. Interface and Release Evidence

For every public RPC, record the tagged SQL definition, generated type, caller grants, parameters, return schema, side effects, transaction boundary, idempotency, failure mapping, and a tested example. For every Edge Function, record deployment status, method, authentication, schemas, CORS behaviour, limits, retries, rate limits, secret names, and audit events. Entries link to an immutable source revision, configuration, and test result; unresolved items remain explicitly unverified.

Documentation is built with:

- `latexmk -pdf -interaction=nonstopmode -halt-on-error main.tex`; and

Before publication, maintainers must:

1. verify repository URLs, the release tag or commit, tool versions, paths, scripts, and configuration against a clean checkout;
2. execute Listings 1–3, generate database types, and retain the test/build result;
3. compare migrations, schema references, RPCs, RLS, storage policies, and export contracts with the deployed release;
4. confirm diagrams and documentation match current component boundaries and workflow states;
5. exercise deployment, forward recovery, backup restoration, privacy requests, and critical troubleshooting in staging;
6. record approved policy and consent versions, retention settings, provider locations, transfer arrangements, and impact-assessment decisions;
7. cross-check pilot facts, model naming, ethics approval, consent rights, limitations, funding, and the external *Jasina* boundary against the companion publication; and

8. compile both documents and resolve undefined citations, references, duplicate labels, and material layout warnings.

Appendix A summarises consequential design decisions. New or superseding decisions retain the context–decision–consequences–alternatives structure in the version-controlled record.

19. Future Extensions

- Expanded language coverage beyond Kikuyu, with administrator-configurable dialect taxonomies.
- Sociolinguistic metadata expansion (region, education, multilingual background) gated by additional consent.
- Optional integration with semi-automated transcription tools (forced alignment, ASR pre-fill) to accelerate annotation.
- Completing and systematically evaluating the separate *Jasina* Kikuyu ASR model on governed JasinaHub exports, including recognition benchmarks and analysis of corpus-quality effects, without coupling model code to the collection platform.
- Pluggable export formats targeting specific machine-learning frameworks (Hugging Face Datasets, Kaldi, NeMo manifests).
- Configurable retirement thresholds and prompt-authoring tools for non-technical curators.
- Contributor engagement analytics including streaks, leaderboards, and lightweight gamification.
- Inter-annotator agreement measurement for transcription consistency using established agreement methodology [4].

These extensions are compatible with the stated architectural direction, but each requires an architecture, privacy, migration, and operational-impact review before implementation.

20. Conclusion

JasinaHub provides an end-to-end workflow for structured speech collection, approved transcription, two-stage quality review, interrupted-upload recovery, and administrator-controlled export. The Kikuyu pilot demonstrated this workflow under practical deployment conditions and produced 1,789 completed speech–transcription pairs, while the limitations in Section 17.1 constrain broader claims.

Future work will extend the platform to additional language communities, investigate semi-automated transcription, evaluate the separate *Jasina* ASR model, and quantify transcription consistency. Maintainers must keep this guide aligned with the released code, migrations, configuration, privacy controls, operational procedures, and test evidence.

21. Project Attribution and Declarations

CRedit authorship contribution statement. **Braxton Mandara:** Conceptualization, Software, Writing—original draft. **Mary Kariuki:** Supervision, Validation. **Ciira wa Maina:** Conceptualization, Project administration, Writing—review and editing.

Ethics statement. The Dedan Kimathi University of Technology Scientific Ethics Review Committee reviewed and approved the study (approval number DeKUT/ISREC/03422/109). All participants provided informed electronic consent before data collection, as described in Section 16.

Declaration of competing interest. The authors declare that they have no known competing financial interests or personal relationships that might have influenced the work reported in this document.

22. Acknowledgments

This work was conducted as part of the Artificial Intelligence for Development (AI4D) program, with the financial support of the UK government through its Global Research and Technology Development portfolio as well as Canada’s International Development Research Centre (IDRC).

A. Architecture Decision Summary

The following summary records the accepted v1.0 decisions. Full ADRs remain in version control with context, alternatives, consequences, dates, and links to superseding decisions.

ADR	Decision	Principal consequence
001	Use Supabase as the Backend-as-a-Service platform.	Authentication, PostgreSQL, Realtime, storage, and Edge Functions reduce server-management overhead but introduce provider-specific configuration and migration concerns.
002	Store privileged roles in <code>user_roles</code> .	Role mutation is separated from editable profiles; <code>has_role()</code> requires safe ownership, <code>search_path</code> , grants, and tests.
003	Model transcription as an approved profile capability.	One account can record and transcribe, but authorisation must combine verification, approval, and guideline acceptance.
004	Use an exclusive configurable transcription lease.	Atomic reservation prevents double assignment and permits expiry; submission must verify the lease and slow users require draft recovery.
005	Capture WebM/Opus with a configurable duration limit.	Browser capture is compact, but deployments must test MIME support and record the actual format in exports.
006	Retain one bounded recovery copy in <code>localStorage</code> .	Reload recovery is simple but synchronous, quota-limited, and unsuitable for a general queue; IndexedDB is the expansion path.
007	Retire prompts at a configurable unique-response threshold.	Collection favours prompt breadth, while backend enforcement and an effective-date history handle concurrency and configuration changes.
008	Store feedback without a direct <code>user_id</code> .	Direct linkage is reduced, but timestamps and infrastructure logs may still permit correlation.
009	Restrict export to administrators and use <code>JSN-XXXXXXX</code> pseudonyms.	Governance and same-speaker linkage are preserved, but voice and metadata remain potentially identifying.
010	Support spontaneous and read-speech prompts in one pipeline.	Collection modes share infrastructure, but mode-specific instructions, review, and metadata remain explicit.

B. Glossary

Accepted recording. Audio that passed administrator review and is eligible for transcription.

Active lock. An unexpired lease granting one approved transcriber temporary responsibility for a recording.

Approved transcriber. A verified volunteer with transcription approval who accepted the guidelines.

Canonical transcription. The single authoritative transcript associated with a recording.

Dataset export. An administrator-authorised package of eligible audio, a manifest, and dataset-level metadata.

Metadata-only export. An explicitly incomplete fallback produced when one or more audio objects cannot be retrieved.

Prompt. Text that elicits a spontaneous response or supplies a predetermined read-speech sentence.

Prompt retirement. Removal of a prompt from active rotation after the configured number of unique contributors respond.

Pseudonymous contributor identifier. A stable `JSN-XXXXXXX` export identifier that replaces a name without making the voice anonymous.

Recovery copy. One bounded Base64 recording retained through `recordingStorage` in `localStorage` until server confirmation or user deletion.

Transcription validation. Administrator comparison of submitted text with the associated audio before export eligibility.

Jasina. The separate Kikuyu ASR project that consumes governed JasinaHub exports; it is outside the platform boundary.

Verified user. An authenticated user approved for protected collection workflows.

Voice response. The database record and audio object created from a submitted spoken answer.

References

- [1] Benjamin Aker, Evelyn Nafula Ouma, Gilbert Yiga, Patrick Walukagga, Phionah Natukunda, Trevor Saaka, Solomon Nsumba, Lilian Teddy Nabukeera, Joel Tibabwetiza Muhanguzi, Imran Sekalala, Nimpamya Janat Namara, Engineer Bainomugisha, Ernest Mwebaze, and John Quinn. Sunflower: A new approach to expanding coverage of african languages in large language models. In *Proceedings of the AI for African Languages Conference 2025*, volume 314 of *Proceedings of Machine Learning Research*, pages 1–20, 2026. URL <https://proceedings.mlr.press/v314/akera26a.html>.
- [2] Benjamin Aker, Patrick Walukagga, Gilbert Yiga, John Quinn, and Ernest Mwebaze. How much speech data is necessary for asr in african languages? an evaluation of data scaling in kinyarwanda and kikuyu. In *Proceedings of the AI for African Languages Conference 2025*, volume 314 of *Proceedings of Machine Learning Research*, pages 41–49, 2026. URL <https://proceedings.mlr.press/v314/akera26b.html>.
- [3] Rosana Ardila, Megan Branson, Kelly Davis, Michael Henretty, Michael Kohler, Josh Meyer, Reuben Morais, Lindsay Saunders, Francis M. Tyers, and Gregor Weber. Common voice: A massively-multilingual speech corpus. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 4218–4222, 2020. URL <https://aclanthology.org/2020.lrec-1.520/>.
- [4] Ron Artstein and Massimo Poesio. Inter-coder agreement for computational linguistics. *Computational Linguistics*, 34(4):555–596, 2008. doi: 10.1162/coli.07-034-R2. URL <https://aclanthology.org/J08-4004/>.
- [5] Arun Babu, Changhan Wang, Andros Tjandra, Kushal Lakhotia, Qiantong Xu, Naman Goyal, Kritika Singh, et al. XLS-R: Self-supervised cross-lingual speech representation learning at scale. In *Interspeech 2022*, pages 2278–2282, 2022. doi: 10.21437/Interspeech.2022-143. URL https://www.isca-archive.org/interspeech_2022/babu22_interspeech.html.
- [6] Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. In *Advances in Neural Information Processing Systems*, volume 33, pages 12449–12460, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/92d1e1eb1cd6f9fba3227870bb6d7f07-Abstract.html>.
- [7] Etienne Barnard, Marelise Davel, and Charl van Heerden. The lwazi project: Speech-based access to information for south african languages. *South African Computer Journal*, 42:1–16, 2009.
- [8] Steven Bird, Florian R. Hanke, Oliver Adams, and Haejoong Lee. Aikuma: A mobile app for collaborative language documentation. In *Proceedings of the 2014 Workshop on the Use of Computational Methods in the Study of Endangered Languages*, pages 1–5, 2014. doi: 10.3115/v1/W14-2201. URL <https://aclanthology.org/W14-2201/>.
- [9] Alexis Conneau, Min Ma, Simran Khanuja, Yu Zhang, Vera Axelrod, Siddharth Dalmia, Jason Riesa, Clara Rivera, and Ankur Bapna. FLEURS: Few-shot learning evaluation of universal representations of speech. In *2022 IEEE Spoken Language Technology Workshop*, pages 798–805, 2023. doi: 10.1109/SLT54892.2023.10023141. URL <https://doi.org/10.1109/SLT54892.2023.10023141>.
- [10] Nicole Davila, Igor Wiese, Igor Steinmacher, Lucas Lucio da Silva, Andre Kawamoto, Gilson Jose Peres Favaro, and Ingrid Nunes. An industry case study on adoption of AI-based programming assistants. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice*, pages 92–102. Association for Computing Machinery, 2024. doi: 10.1145/3639477.3643648.
- [11] David M. Eberhard, Gary F. Simons, and Charles D. Fennig. *Ethnologue: Languages of the World*. SIL International, Dallas, Texas, 28 edition, 2025. URL <https://www.ethnologue.com/>.
- [12] AbdelRahim A. Elmadany, Sang Yun Kwon, Hawau Olamide Toyin, Alcides Alcoba Inciarte, Hanan Aldarmaki, and Muhammad Abdul-Mageed. Voice of a continent: Mapping Africa’s speech technology frontier. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 11028–11050. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.559. URL <https://aclanthology.org/2025.emnlp-main.559/>.

- [13] Elodie Gauthier, Laurent Besacier, Sylvie Voisin, Michael Melese, and Uriel Pascal Elingui. Collecting resources in sub-saharan african languages for automatic speech recognition: A case study of Wolof. In *Proceedings of the Tenth International Conference on Language Resources and Evaluation*, pages 3863–3867, 2016. URL <https://aclanthology.org/L16-1611/>.
- [14] Elodie Gauthier, David Blachon, Laurent Besacier, Guy-Noël Kouarata, Martine Adda-Decker, Annie Rialland, Gilles Adda, and Grégoire Bachman. LIG-Aikuma: A mobile app to collect parallel speech for under-resourced language studies. In *Interspeech 2016*, pages 381–382, 2016. URL https://www.isca-archive.org/interspeech2016/gauthier16_interspeech.html.
- [15] Jim Gray. The transaction concept: Virtues and limitations. In *Proceedings of the Seventh International Conference on Very Large Data Bases*, pages 144–154, 1981.
- [16] Joseph M. Hellerstein, Jose Faleiro, Joseph E. Gonzalez, Johann Schleier-Smith, Vikram Sreekanti, Alexey Tumanov, and Chenggang Wu. Serverless computing: One step forward, two steps back. In *Proceedings of the 9th Biennial Conference on Innovative Data Systems Research*, 2019. URL <https://www.cidrdb.org/cidr2019/papers/p119-hellerstein-cidr19.pdf>.
- [17] S. H. Imam, B. Sani, D. K. Gete, B. Y. Ahamed, I. S. Ahmad, I. Abdulmumin, S. M. Yimam, M. Y. Bello, and S. H. Muhammad. Automatic speech recognition for african low-resource languages: Challenges and future directions. *arXiv preprint arXiv:2505.11690*, 2025. doi: 10.48550/arXiv.2505.11690. URL <https://arxiv.org/abs/2505.11690>.
- [18] Kenya National Bureau of Statistics. 2019 kenya population and housing census, volume iv: Distribution of population by socio-economic characteristics. Technical report, Kenya National Bureau of Statistics, Nairobi, Kenya, 2019. URL <https://www.knbs.or.ke/reports/kenya-census-2019/>.
- [19] Jonathan Mukiibi, Andrew Katumba, Joyce Nakatumba-Nabende, Ali Hussein, and Joshua Meyer. The makerere radio speech corpus: A luganda radio corpus for automatic speech recognition. In *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pages 1945–1954, 2022. URL <https://aclanthology.org/2022.lrec-1.208/>.
- [20] Wilhelmina Nekoto, Vukosi Marivate, Tshinondiwa Matsila, Timi Fasubaa, Taiwo Fagbohunge, Solomon Oluwole Akinola, Shamsuddeen Hassan Muhammad, et al. Participatory research for low-resourced machine translation: A case study in african languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2144–2160, 2020. doi: 10.18653/v1/2020.findings-emnlp.195. URL <https://aclanthology.org/2020.findings-emnlp.195/>.
- [21] Vineel Pratap, Qiantong Xu, Anuroop Sriram, Gabriel Synnaeve, and Ronan Collobert. MLS: A large-scale multilingual dataset for speech research. In *Interspeech 2020*, pages 2757–2761, 2020. doi: 10.21437/Interspeech.2020-2826. URL https://www.isca-archive.org/interspeech2020/pratap20_interspeech.html.
- [22] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 28492–28518, 2023. URL <https://proceedings.mlr.press/v202/radford23a.html>.
- [23] Republic of Kenya. Data protection act, no. 24 of 2019. Kenya Law, 2019. URL <https://new.kenyalaw.org/akn/ke/act/2019/24/eng@2022-12-31>.
- [24] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. *Computer*, 29(2):38–47, 1996. doi: 10.1109/2.485845.
- [25] Tanja Schultz. Globalphone: A multilingual speech and text database developed at karlsruhe university. In *Proceedings of the 7th International Conference on Spoken Language Processing*, pages 345–348, 2002.
- [26] Natalia Tomashenko, Xin Wang, Emmanuel Vincent, Jose Patino, Brij Mohan Lal Srivastava, Paul-Gauthier Noé, Andreas Nautsch, Nicholas Evans, et al. The VoicePrivacy 2020 challenge: Results and findings. *Computer Speech & Language*, 74:101362, 2022. doi: 10.1016/j.csl.2022.101362. URL <https://doi.org/10.1016/j.csl.2022.101362>.

- [27] Jean-Marc Valin, Koen Vos, and Timothy B. Terriberry. Definition of the opus audio codec. RFC 6716, Internet Engineering Task Force, 2012. URL <https://www.rfc-editor.org/rfc/rfc6716>.
- [28] Barack Wanjawa, Lilian Wanzare, Florence Indede, Owen McOnyango, Edward Ombui, and Lawrence Muchemi. Kencorpus: A kenyan language corpus of swahili, dholuo and luhya for natural language processing tasks. *Journal for Language Technology and Computational Linguistics*, 36(2):1–27, 2023. URL <https://aclanthology.org/2023.jlcl-2.1/>.
- [29] Lilian Wanzare, Cynthia Amol, Ezekiel Maina, Nelson Odhiambo, Hope Kerubo, Leila Misula, Vivian Oloo, Rennish Mboya, Edwin Onkoba, Edward Ombui, Joseph Muguro, Ciira wa Maina, Andrew Kipkebut, Alfred Omondi Otom, Ian Ndung’u Kang’ethe, Angela Wambui Kanyi, and Brian Gichana Omwenga. AfriVoices-KE: A multilingual speech dataset for kenyan languages. *arXiv preprint arXiv:2604.08448*, 2026. URL <https://arxiv.org/abs/2604.08448>.
- [30] World Wide Web Consortium. *MediaStream Recording*. W3C, 2026. URL <https://www.w3.org/TR/mediastream-recording/>. Working Draft, 16 March 2026.